



localhost:8888/nbconvert/html/OneDrive/IBM Data Science Professional Certificate/Course materials%2C practices and projects/9. Machine Learning... 1/40

About The Dataset

The original source of the data is Australian Government's Bureau of Meteorology and the latest data can be gathered from <http://www.bom.gov.au/climate/dwo/> (http://www.bom.gov.au/climate/dwo/?utm_medium=Exinfluencer&utm_source=Exinfluencer&utm_content=000026UJ&utm_term=10006555&utm_id=NA&SkillsNetwork-Channel-SkillsNetworkCoursesIBMDeveloperSkillsNetworkML0101ENSkillsNetwork20718538-2022-01-01).

The dataset you'll use in this project was downloaded from Kaggle at <https://www.kaggle.com/datasets/jsphyg/weather-dataset-rattle-package/> (<https://www.kaggle.com/datasets/jsphyg/weather-dataset-rattle-package?resource=download&select=weatherAUS.csv>).

Column definitions were gathered from <http://www.bom.gov.au/climate/dwo/IDCJDW0000.shtml> (http://www.bom.gov.au/climate/dwo/IDCJDW0000.shtml?utm_medium=Exinfluencer&utm_source=Exinfluencer&utm_content=000026UJ&utm_term=10006555&utm_id=NA&SkillsNetwork-Channel-SkillsNetworkCoursesIBMDeveloperSkillsNetworkML0101ENSkillsNetwork20718538-2022-01-01).

The dataset contains observations of weather metrics for each day from 2008 to 2017, and includes the following fields:

Field	Description	Unit	Type
Date	Date of the Observation in YYYY-MM-DD	Date	object
Location	Location of the Observation	Location	object
MinTemp	Minimum temperature	Celsius	float
MaxTemp	Maximum temperature	Celsius	float
Rainfall	Amount of rainfall	Millimeters	float
Evaporation	Amount of evaporation	Millimeters	float
Sunshine	Amount of bright sunshine	hours	float
WindGustDir	Direction of the strongest gust	Compass Points	object
WindGustSpeed	Speed of the strongest gust	Kilometers/Hour	object
WindDir9am	Wind direction averaged over 10 minutes prior to 9am	Compass Points	object
WindDir3pm	Wind direction averaged over 10 minutes prior to 3pm	Compass Points	object
WindSpeed9am	Wind speed averaged over 10 minutes prior to 9am	Kilometers/Hour	float
WindSpeed3pm	Wind speed averaged over 10 minutes prior to 3pm	Kilometers/Hour	float
Humidity9am	Humidity at 9am	Percent	float
Humidity3pm	Humidity at 3pm	Percent	float
Pressure9am	Atmospheric pressure reduced to mean sea level at 9am	Hectopascal	float
Pressure3pm	Atmospheric pressure reduced to mean sea level at 3pm	Hectopascal	float
Cloud9am	Fraction of the sky obscured by cloud at 9am	Eights	float
Cloud3pm	Fraction of the sky obscured by cloud at 3pm	Eights	float

Field	Description	Unit	Type
Temp9am	Temperature at 9am	Celsius	float
Temp3pm	Temperature at 3pm	Celsius	float
RainToday	If there was at least 1mm of rain today	Yes/No	object
RainTomorrow	If there is at least 1mm of rain tomorrow	Yes/No	object

Install and import the required libraries

Execute the following cells to install and import the necessary libraries.

```
In [1]: # !pip install numpy
        # !pip install pandas
        # !pip install matplotlib
        # !pip install scikit-learn
        # !pip install seaborn
```

```
In [328]: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
# !pip install -U scikit-learn
# from sklearn.metrics import ConfusionMatrixDisplay
import seaborn as sns
```

Load the data

Execute the following cells to load the dataset as a pandas dataframe.

```
In [329]: url="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/_0eY0q
          ji3unP1tDNKWMjg/weatherAUS-2.csv"
          df = pd.read_csv(url)
          df.head()
```

Out[329]:

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	WindGust
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	W	
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	WNW	
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	WSW	
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	NE	
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	W	

5 rows × 23 columns



```
In [246]: df.count()
```

```
Out[246]: Date          145460
Location        145460
MinTemp         143975
MaxTemp         144199
Rainfall        142199
Evaporation      82670
Sunshine        75625
WindGustDir     135134
WindGustSpeed   135197
WindDir9am      134894
WindDir3pm      141232
WindSpeed9am    143693
WindSpeed3pm    142398
Humidity9am     142806
Humidity3pm     140953
Pressure9am     130395
Pressure3pm     130432
Cloud9am        89572
Cloud3pm        86102
Temp9am         143693
Temp3pm         141851
RainToday       142199
RainTomorrow    142193
dtype: int64
```



```
In [330]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145460 entries, 0 to 145459
Data columns (total 23 columns):
Date                145460 non-null object
Location            145460 non-null object
MinTemp             143975 non-null float64
MaxTemp             144199 non-null float64
Rainfall            142199 non-null float64
Evaporation          82670 non-null float64
Sunshine            75625 non-null float64
WindGustDir          135134 non-null object
WindGustSpeed        135197 non-null float64
WindDir9am           134894 non-null object
WindDir3pm           141232 non-null object
WindSpeed9am         143693 non-null float64
WindSpeed3pm         142398 non-null float64
Humidity9am          142806 non-null float64
Humidity3pm          140953 non-null float64
Pressure9am          130395 non-null float64
Pressure3pm          130432 non-null float64
Cloud9am             89572 non-null float64
Cloud3pm             86102 non-null float64
Temp9am              143693 non-null float64
Temp3pm              141851 non-null float64
RainToday            142199 non-null object
RainTomorrow         142193 non-null object
dtypes: float64(16), object(7)
memory usage: 25.5+ MB
```

Sunshine and cloud cover seem like important features, but they have a lot of missing values, far too many to impute their missing values.

Drop all rows with missing values

To try to keep things simple we'll drop rows with missing values and see what's left

```
In [331]: df = df.dropna()
df.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 56420 entries, 6049 to 142302
Data columns (total 23 columns):
Date                56420 non-null object
Location            56420 non-null object
MinTemp             56420 non-null float64
MaxTemp             56420 non-null float64
Rainfall            56420 non-null float64
Evaporation         56420 non-null float64
Sunshine            56420 non-null float64
WindGustDir         56420 non-null object
WindGustSpeed       56420 non-null float64
WindDir9am          56420 non-null object
WindDir3pm          56420 non-null object
WindSpeed9am        56420 non-null float64
WindSpeed3pm        56420 non-null float64
Humidity9am         56420 non-null float64
Humidity3pm         56420 non-null float64
Pressure9am         56420 non-null float64
Pressure3pm         56420 non-null float64
Cloud9am            56420 non-null float64
Cloud3pm            56420 non-null float64
Temp9am             56420 non-null float64
Temp3pm             56420 non-null float64
RainToday           56420 non-null object
RainTomorrow        56420 non-null object
dtypes: float64(16), object(7)
memory usage: 10.3+ MB
```

```
In [252]: # correlation_matrix = df.corr()

# print(correlation_matrix)
```

Since we still have 56k observations left after dropping missing values, we may not need to impute any missing values.

Let's see how we do.

```
In [332]: df.columns

Out[332]: Index(['Date', 'Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation',
                'Sunshine', 'WindGustDir', 'WindGustSpeed', 'WindDir9am', 'WindDir3p
                m',
                'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm',
                'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Cloud3pm', 'Temp9am',
                'Temp3pm', 'RainToday', 'RainTomorrow'],
                dtype='object')
```

Data leakage considerations

Consider the descriptions above for the columns in the data set. Are there any practical limitations to being able to predict whether it will rain tomorrow given the available data?

Points to note - 1

List some of the features that would be inefficient in predicting tomorrow's rainfall. There will be a question in the quiz that follows based on this observation.

When predicting tomorrow's rainfall, some features might be inefficient, especially if they rely on the entire duration of today for their evaluation. Here are some features from your list that might be less effective:

Date: The specific date is unlikely to have a direct impact on rainfall unless it is used to derive seasonal information, which would be more relevant.

Location: While location can influence climate patterns, it is a categorical variable that might not provide direct predictive power without additional context or transformation.

WindGustDir: Wind direction during gusts might not be as directly related to rainfall as other wind-related features.

WindDir9am and WindDir3pm: Similar to WindGustDir, the wind direction at specific times might not be as predictive as wind speed or humidity.**

[Click here for Hint](#)

If we adjust our approach and aim to predict today's rainfall using historical weather data up to and including yesterday, then we can legitimately utilize all of the available features. This shift would be particularly useful for practical applications, such as deciding whether you will bike to work today.

With this new target, we should update the names of the rain columns accordingly to avoid confusion.

```
In [333]: df = df.rename(columns={'RainToday': 'RainYesterday',
                                'RainTomorrow': 'RainToday'})
df.columns
```

```
Out[333]: Index(['Date', 'Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'Evaporation',
                'Sunshine', 'WindGustDir', 'WindGustSpeed', 'WindDir9am', 'WindDir3p
                m',
                'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm',
                'Pressure9am', 'Pressure3pm', 'Cloud9am', 'Cloud3pm', 'Temp9am',
                'Temp3pm', 'RainYesterday', 'RainToday'],
                dtype='object')
```

Data Granularity

Would the weather patterns have the same predictability in vastly different locations in Australia? I would think not.

The chance of rain in one location can be much higher than in another. Using all of the locations requires a more complex model as it needs to adapt to local weather patterns.

Let's see how many observations we have for each location, and see if we can reduce our attention to a smaller region.

Location selection

You could do some research to group cities in the `Location` column by distance, which I've done for you behind the scenes.

I found that Watsonia is only 15 km from Melbourne, and the Melbourne Airport is only 18 km from Melbourne.

Let's group these three locations together and use only their weather data to build our localized prediction model.

Because there might still be some slight variations in the weather patterns we'll keep `Location` as a categorical variable.

```
In [334]: df = df[df.Location.isin(['Melbourne', 'MelbourneAirport', 'Watsonia', ])]
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 7557 entries, 64191 to 80997
Data columns (total 23 columns):
Date                7557 non-null object
Location            7557 non-null object
MinTemp             7557 non-null float64
MaxTemp             7557 non-null float64
Rainfall            7557 non-null float64
Evaporation         7557 non-null float64
Sunshine            7557 non-null float64
WindGustDir         7557 non-null object
WindGustSpeed       7557 non-null float64
WindDir9am          7557 non-null object
WindDir3pm          7557 non-null object
WindSpeed9am        7557 non-null float64
WindSpeed3pm        7557 non-null float64
Humidity9am         7557 non-null float64
Humidity3pm         7557 non-null float64
Pressure9am         7557 non-null float64
Pressure3pm         7557 non-null float64
Cloud9am            7557 non-null float64
Cloud3pm            7557 non-null float64
Temp9am             7557 non-null float64
Temp3pm             7557 non-null float64
RainYesterday       7557 non-null object
RainToday           7557 non-null object
dtypes: float64(16), object(7)
memory usage: 1.4+ MB
```

```
In [255]: df.head()
```

Out[255]:

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir
64191	2009-01-01	MelbourneAirport	11.2	19.9	0.0	5.6	8.8	SW
64192	2009-01-02	MelbourneAirport	7.8	17.8	1.2	7.2	12.9	SSE
64193	2009-01-03	MelbourneAirport	6.3	21.1	0.0	6.2	10.5	SSE
64194	2009-01-04	MelbourneAirport	8.1	29.2	0.0	6.4	12.5	SSE
64195	2009-01-05	MelbourneAirport	9.7	29.0	0.0	7.4	12.3	SE

5 rows × 23 columns



```
In [335]: # Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Extract the unique years from the 'Date' column
unique_years = df['Date'].dt.year.nunique()

# Print the total number of years
print(f"The data spans across {unique_years} unique years.")
```

The data spans across 10 unique years.

We still have 7557 records, which should be enough to build a reasonably good model.

You could always gather more data if needed by partitioning the data into similar locations or simply by updating it from the source to include a larger time frame.

Extracting a seasonality feature

Now consider the `Date` column. We expect the weather patterns to be seasonal, having different predictability levels in winter and summer for example.

There may be some variation with `Year` as well, but we'll leave that out for now. Let's engineer a `Season` feature from `Date` and drop `Date` afterward, since it is most likely less informative than season. An easy way to do this is to define a function that assigns seasons to given months, then use that function to transform the `Date` column.

Create a function to map dates to seasons

```
In [336]: def date_to_season(date):
month = date.month
if (month == 12) or (month == 1) or (month == 2):
    return 'Summer'
elif (month == 3) or (month == 4) or (month == 5):
    return 'Autumn'
elif (month == 6) or (month == 7) or (month == 8):
    return 'Winter'
elif (month == 9) or (month == 10) or (month == 11):
    return 'Spring'
```

In [260]: `df.head()`

Out[260]:

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir
64191	2009-01-01	MelbourneAirport	11.2	19.9	0.0	5.6	8.8	SW
64192	2009-01-02	MelbourneAirport	7.8	17.8	1.2	7.2	12.9	SSE
64193	2009-01-03	MelbourneAirport	6.3	21.1	0.0	6.2	10.5	SSE
64194	2009-01-04	MelbourneAirport	8.1	29.2	0.0	6.4	12.5	SSE
64195	2009-01-05	MelbourneAirport	9.7	29.0	0.0	7.4	12.3	SE

5 rows × 23 columns



Exercise 1: Map the dates to seasons and drop the Date column

Complete the code:

```
# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(...)
```

```
# Apply the function to the 'Date' column
df['Season'] = df['Date'].apply(date_to_season)
```

```
df=df.drop(columns=...)
df
```

```
In [337]: # Write your response.
# Convert the 'Date' column to datetime format
df['Date'] = pd.to_datetime(df['Date'])

# Apply the function to the 'Date' column
df['Season'] = df['Date'].apply(date_to_season)

# Drop the 'Date' column from the DataFrame
df = df.drop(columns=['Date'])
df
```

Out[337]:

	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGustDir	Wind
64191	MelbourneAirport	11.2	19.9	0.0	5.6	8.8	SW	
64192	MelbourneAirport	7.8	17.8	1.2	7.2	12.9	SSE	
64193	MelbourneAirport	6.3	21.1	0.0	6.2	10.5	SSE	
64194	MelbourneAirport	8.1	29.2	0.0	6.4	12.5	SSE	
64195	MelbourneAirport	9.7	29.0	0.0	7.4	12.3	SE	
...	...	...	...	...	...	...	...	...
80992	Watsonia	3.6	14.5	0.0	2.4	8.8	NNE	
80994	Watsonia	4.8	13.3	0.4	0.6	0.0	NNW	
80995	Watsonia	5.6	13.1	0.0	1.6	6.0	NNW	
80996	Watsonia	6.9	12.1	3.2	1.8	5.6	SSW	
80997	Watsonia	7.9	13.0	0.0	2.8	3.8	NNW	

7557 rows × 23 columns



Looks like we have a good set of features to work with.

Let's go ahead and build our model.

But wait, let's take a look at how well balanced our target is.

Exercise 2. Define the feature and target dataframes

Complete the following code:

```
X = df.drop(columns='...', axis=1)
y = df['...']
```



```
In [338]: # Write your response.
X = df.drop(columns='RainToday', axis=1)
y = df['RainToday']
print(X.shape)
print(y.shape)

(7557, 22)
(7557,)
```

Exercise 3. How balanced are the classes?

Display the counts of each class.

Complete the following code:

```
... .value_counts()
```

```
In [339]: print(y.value_counts())
rain_today_counts_mel_area = y.value_counts()/unique_years
print(f"Melbourne Area Rain BY YEAR: No Rain: {rain_today_counts_mel_area['No']}, Yes Rain: {rain_today_counts_mel_area['Yes']}")

No      5766
Yes     1791
Name: RainToday, dtype: int64
Melbourne Area Rain BY YEAR: No Rain: 576.6, Yes Rain: 179.1
```

Exercise 4. What can you conclude from these counts?

How often does it rain annually in the Melbourne Area? 179 days per year based on this 10-year time span

How accurate would you be if you just assumed it won't rain every day?
Accuracy=(5766/7557)X100=76.3%

Is this a balanced dataset? It is not a balanced dataset. The ratio of "No" to "Yes" is approximately 3.1. This indicates that the dataset is imbalanced, with "No rain" significantly outnumbering "Yes rain."

What is next: balance the dataset and examine distribution

Breaking down the dataset by Unique Areas

```
In [340]: df_melbourne = df[df['Location'] == 'Melbourne']
df_melbourne_airport = df[df['Location'] == 'MelbourneAirport']
df_melbourne_watsonia = df[df['Location'] == 'Watsonia']
```

```
In [341]: location_counts = df.groupby('Location').size().reset_index(name='Counts')
print(location_counts)

total_counts = location_counts['Counts'].sum()

print("\nTotal counts:", total_counts)
```

	Location	Counts
0	Melbourne	1898
1	MelbourneAirport	2929
2	Watsonia	2730

Total counts: 7557

```
In [342]: # Write your response.
rain_today_counts_mel = df_melbourne['RainToday'].value_counts()/unique_years
rain_today_counts_mel_air = df_melbourne_airport['RainToday'].value_counts()/unique_years
rain_today_counts_watsonia = df_melbourne_watsonia['RainToday'].value_counts()/unique_years

print(f"Melbourne Rain BY YEAR: No Rain: {rain_today_counts_mel['No']}, Yes Rain: {rain_today_counts_mel['Yes']}")
print(f"Melbourne Airport Rain BY YEAR: No Rain: {rain_today_counts_mel_air['No']}, Yes Rain: {rain_today_counts_mel_air['Yes']}")
print(f"Watsonia Rain BY YEAR: No Rain: {rain_today_counts_watsonia['No']}, Yes Rain: {rain_today_counts_watsonia['Yes']}")
```

Melbourne Rain BY YEAR: No Rain: 142.7, Yes Rain: 47.1
 Melbourne Airport Rain BY YEAR: No Rain: 229.3, Yes Rain: 63.6
 Watsonia Rain BY YEAR: No Rain: 204.6, Yes Rain: 68.4

In []:

Exercise 4. What can you conclude from these counts?

- How often does it rain annually in the Melbourne area?
- How accurate would you be if you just assumed it won't rain every day?
- Is this a balanced dataset?
- Next steps?

Write your response here and convert the cell to a markdown.

How often does it rain annually in the Melbourne area? 47.1 days per year

How accurate would you be if you just assumed it won't rain every day? $\text{Accuracy} = (1427/1898) \times 100 = 75.2\%$

Is this a balanced dataset? It is not a balanced dataset. The ratio of "No" to "Yes" is approximately 3.1. This indicates that the dataset is imbalanced, with "No rain" significantly outnumbering "Yes rain."

What is next: balance the dataset and examine distribution

Exercise 5. Split data into training and test sets, ensuring target stratification

Complete the following code:

```
X_train, X_test, y_train, y_test = train_test_split(..., ..., test_size=0.2, stratify=..., random_state=42)
```

```
In [343]: # Write your response.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
```

Define preprocessing transformers for numerical and categorical features

Exercise 6. Automatically detect numerical and categorical columns and assign them to separate numeric and categorical features

Complete the following code:

```
numeric_features = X_train.select_dtypes(include=['...']).columns.tolist()
categorical_features = X_train.select_dtypes(include=['...', 'category']).columns.tolist()
```

```
In [344]: # Write your response.
numerical_features = X_train.select_dtypes(include=['number']).columns.tolist()
categorical_features = X_train.select_dtypes(include=['object', 'category']).columns.tolist()
```

Define separate transformers for both feature types and combine them into a single preprocessing transformer

```
In [345]: # Scale the numeric features
numeric_transformer = Pipeline(steps=[('scaler', StandardScaler())])

# One-hot encode the categoricals
categorical_transformer = Pipeline(steps=[('onehot', OneHotEncoder(handle_unknown='ignore'))])
```

Exercise 7. Combine the transformers into a single preprocessing column transformer

Complete the following code:

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, ...),
        ('cat', categorical_transformer, ...)
    ]
)
```

```
In [346]: # Write your response.
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)
preprocessor
```

```
Out[346]: ColumnTransformer(n_jobs=None, remainder='drop', sparse_threshold=0.3,
                             transformer_weights=None,
                             transformers=[('num',
                                           Pipeline(memory=None,
                                                  steps=[('scaler',
                                                         StandardScaler(copy=True,
                                                                    with_mean=True,
                                                                    with_std=True,
                                                                    dtype='numpy.float64',
                                                                    handle_unknown='ignore',
                                                                    n_values=None,
                                                                    sparse=True,
                                                                    verbose=False),
                                                         ['MinTemp', 'MaxTemp', 'Rainfall',
                                                          'Evaporation', 'Sunshine', 'WindGustSpeed',
                                                          'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am',
                                                          'Humidity3pm', 'Cloud9am', 'Cloud3pm', 'Temp9am',
                                                          'Temp3pm']),
                                                         ('cat',
                                                          Pipeline(memory=None,
                                                                 steps=[('onehot',
                                                                           OneHotEncoder(categorical_f
                                                                           categories=None,
                                                                           drop=None,
                                                                           dtype=<class
                                                                           handle_unknown='ignore',
                                                                           n_values=None,
                                                                           sparse=True,
                                                                           verbose=False),
                                                                           ['Location', 'WindGustDir', 'WindDir9am',
                                                                           'WindDir3pm', 'RainYesterday', 'Season'])],
                                                                           verbose=False)
```

Exercise 8. Create a pipeline by combining the preprocessing with a Random Forest classifier

Complete the following code:

```
pipeline = Pipeline(steps=[
    ('preprocessor', ...),
    ('...', RandomForestClassifier(random_state=42))
])
```

```
In [347]: # Write your response.
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(random_state=42))
])
```

Define a parameter grid to use in a cross validation grid search model optimizer

```
In [348]: param_grid = {
    'classifier__n_estimators': [50, 100],
    'classifier__max_depth': [None, 10, 20],
    'classifier__min_samples_split': [2, 5]
}
```

Pipeline usage in crossvalidation

Recall that the pipeline is repeatedly used within the crossvalidation by fitting on each internal training fold and predicting on its corresponding validation fold

Perform grid search cross-validation and fit the best model to the training data

Select a cross-validation method, ensuring target stratification during validation

```
In [349]: cv = StratifiedKFold(n_splits=5, shuffle=True)
```

Exercise 9. Instantiate and fit GridSearchCV to the pipeline

Complete the following code:

```
grid_search = GridSearchCV(..., param_grid, cv=..., scoring='accuracy', verbose=2)
grid_search.fit(..., ...)
```

```
In [350]: ### Write your response.  
grid_search = GridSearchCV(estimator=pipeline, param_grid=param_grid, cv=cv, s  
oring='accuracy', verbose=2)  
grid_search.fit(X_train, y_train)
```


Fitting 5 folds for each of 12 candidates, totalling 60 fits

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50

[Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent workers.

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50

[Parallel(n_jobs=1)]: Done 1 out of 1 | elapsed: 0.2s remaining: 0.0s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.6s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.6s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.6s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.6s

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100

[CV] classifier__max_depth=None, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.7s

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.3s

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.4s

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.4s

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.3s

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.4s

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100

[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100

```
__n_estimators=100, total= 0.7s
[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100
[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100, total= 0.6s
[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100
[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100, total= 0.6s
[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100
[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100, total= 0.6s
[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100
[CV] classifier__max_depth=None, classifier__min_samples_split=5, classifier__n_estimators=100, total= 0.7s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.5s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.5s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.5s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.5s
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=2, classifier__n_estimators=100, total= 0.5s
```

[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100, total= 0.5s
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100, total= 0.5s
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100, total= 0.5s
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100, total= 0.5s
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100
[CV] classifier__max_depth=10, classifier__min_samples_split=5, classifier__n_estimators=100, total= 0.5s
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n_estimators=50
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n_estimators=50
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.3s
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n_estimators=50
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.4s
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n_estimators=50
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n_estimators=50, total= 0.4s
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n_estimators=50

```
_estimators=50
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__
n_estimators=50, total= 0.3s
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n
_estimators=100
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__
n_estimators=100, total= 0.7s
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n
_estimators=100
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__
n_estimators=100, total= 0.7s
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n
_estimators=100
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__
n_estimators=100, total= 0.7s
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__n
_estimators=100
[CV] classifier__max_depth=20, classifier__min_samples_split=2, classifier__
n_estimators=100, total= 0.7s
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__n
_estimators=50
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__
n_estimators=50, total= 0.3s
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__n
_estimators=50
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__
n_estimators=50, total= 0.3s
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__n
_estimators=50
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__
n_estimators=50, total= 0.3s
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__n
_estimators=50
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__
n_estimators=50, total= 0.4s
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__n
_estimators=100
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__
n_estimators=100, total= 0.7s
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__n
_estimators=100
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__
n_estimators=100, total= 0.7s
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__n
_estimators=100
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__
n_estimators=100, total= 0.7s
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__n
_estimators=100
```

```
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__
n_estimators=100, total= 0.7s
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__n
_estimators=100
[CV] classifier__max_depth=20, classifier__min_samples_split=5, classifier__
n_estimators=100, total= 0.6s

[Parallel(n_jobs=1)]: Done 60 out of 60 | elapsed: 28.0s finished
```

```
Out[350]: GridSearchCV(cv=StratifiedKFold(n_splits=5, random_state=None, shuffle=True),
                        error_score='raise-deprecating',
                        estimator=Pipeline(memory=None,
                                           steps=[('preprocessor',
                                                  ColumnTransformer(n_jobs=None,
                                                                    remainder='drop',
                                                                    sparse_threshold=0.
3,
                                                                    transformer_weights
=None,
                                                                    transformers=[('nu
m',
                                                         Pipe
line(memory=None,
steps=[('scaler',
StandardScaler(copy=True,
with_mean=Tr...
ction_leaf=0.0,
='warn',
e,
2,
se))),
                                verbose=False),
                        iid='warn', n_jobs=None,
                        param_grid={'classifier__max_depth': [None, 10, 20],
                                   'classifier__min_samples_split': [2, 5],
                                   'classifier__n_estimators': [50, 100]},
                        pre_dispatch='2*n_jobs', refit=True, return_train_score=False,
                        scoring='accuracy', verbose=2)
```

Print the best parameters and best crossvalidation score

```
In [351]: # from sklearn.model_selection import GridSearchCV
# from sklearn.ensemble import RandomForestClassifier
print("\nBest Parameters:", grid_search.best_params_)
print("\nBest Score:", grid_search.best_score_)
```

```
Best Parameters: {'classifier__max_depth': 20, 'classifier__min_samples_split': 5, 'classifier__n_estimators': 50}
```

```
Best Score: 0.8511166253101737
```

Exercise 10. Display your model's estimated score

Complete the following code:

```
test_score = grid_search.score(..., ...)
print("Test set score: {:.2f}".format(test_score))
```

```
In [352]: ## Write your response.
test_score = grid_search.score(X_test, y_test)
print("Test set score: {:.2f}".format(test_score))
```

```
Test set score: 0.85
```

So we have a reasonably accurate classifier, which is expected to correctly predict about 84% of the time whether it will rain today in the Melbourne area.

But careful here. Let's take a deeper look at the results.

The best model is stored within the gridsearch object.

Exercise 11. Get the model predictions from the grid search estimator on the unseen data

Complete the following code:

```
y_pred = grid_search.predict(...)
```

```
In [353]: ### Write your response.
y_pred = grid_search.predict(X_test)
```

Exercise 12. Print the classification report

Complete the following code:

```
print("\nClassification Report:")
print(...(y_test, y_pred))
```

```
In [354]: ## Write your response.
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

     No           0.87        0.95        0.90        1154
     Yes           0.76        0.53        0.62         358

 accuracy              0.85        1512
 macro avg           0.81        0.74        0.76        1512
 weighted avg        0.84        0.85        0.84        1512
```

Exercise 13. Plot the confusion matrix

Complete the following code:

```
conf_matrix = ...(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=...)
disp.plot(cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```



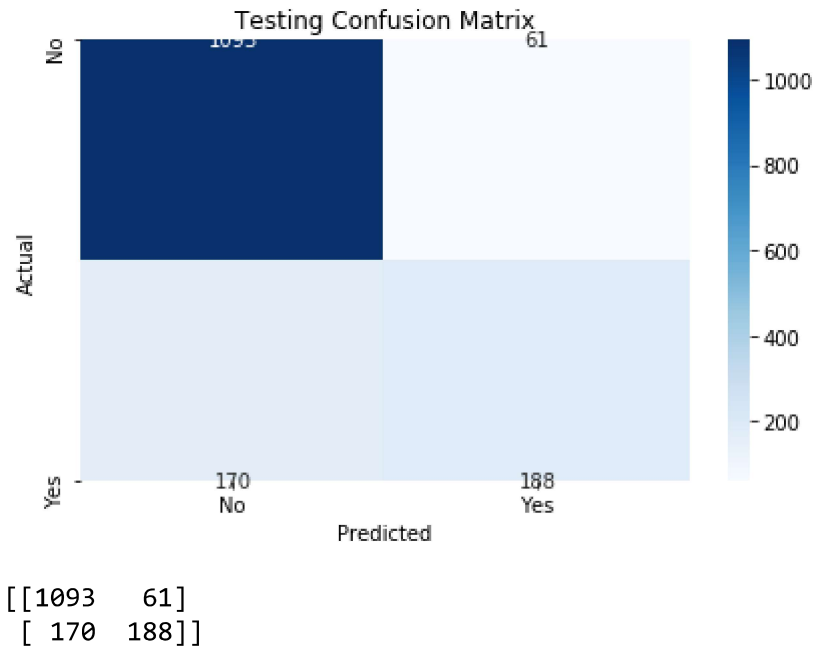
```
In [355]: # Generate the confusion matrix for KNN
conf_matrix = confusion_matrix(y_test, y_pred)

# Create a single plot for the confusion matrix
plt.figure()
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=model.
classes_, yticklabels=model.classes_)

# Set the title and Labels
plt.title('Testing Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')

# Show the plot
plt.tight_layout()
plt.show()

print(conf_matrix)
```



Let's consider whether the results indicate a good predictor of rainfall.

Points to note - 2

What is the true positive rate? There will be a question on this in the assignment that follows.

True positive rate = $187 / (187 + 171) = 187 / 358 = 0.522$. So, 52.2%

```
In [359]: import numpy as np

# Extract True Positives (TP) and False Negatives (FN)
TP = conf_matrix[1, 1]
FN = conf_matrix[1, 0]

# Calculate the True Positive Rate (TPR)
TPR = TP / (TP + FN)

print("True Positive Rate (TPR):", round(TPR*100), "%")
```

True Positive Rate (TPR): 53.0 %

[Click here for Hints](#)

Feature importances

Recall that to obtain the categorical feature importances, we have to work our way backward through the modelling pipeline to associate the feature importances with their original input variables, not the one-hot encoded ones. We don't need to do this for the numeric variables because we didn't modify their names in any way.

Remember we went from categorical features to one-hot encoded features, using the 'cat' column transformer.

Let's get all of the feature importances and associate them with their transformed features

Exercise 14. Extract the feature importances

Complete the following code:

```
feature_importances = grid_search.best_estimator_['classifier']. ...
```

```
In [360]: ## Write your response.
feature_importances = grid_search.best_estimator_['classifier'].feature_importances_

# Combine the numerical and one-hot encoded categorical feature names
feature_names = numerical_features + list(grid_search.best_estimator_['preprocessor'].named_transformers_['cat'].named_steps['onehot'].get_feature_names(categorical_features))
```

Now let's extract the feature importances and plot them as a bar graph.

```
In [361]: importance_df = pd.DataFrame({'Feature': feature_names,
                                       'Importance': feature_importances
                                       }).sort_values(by='Importance', ascending=False)

print(importance_df)

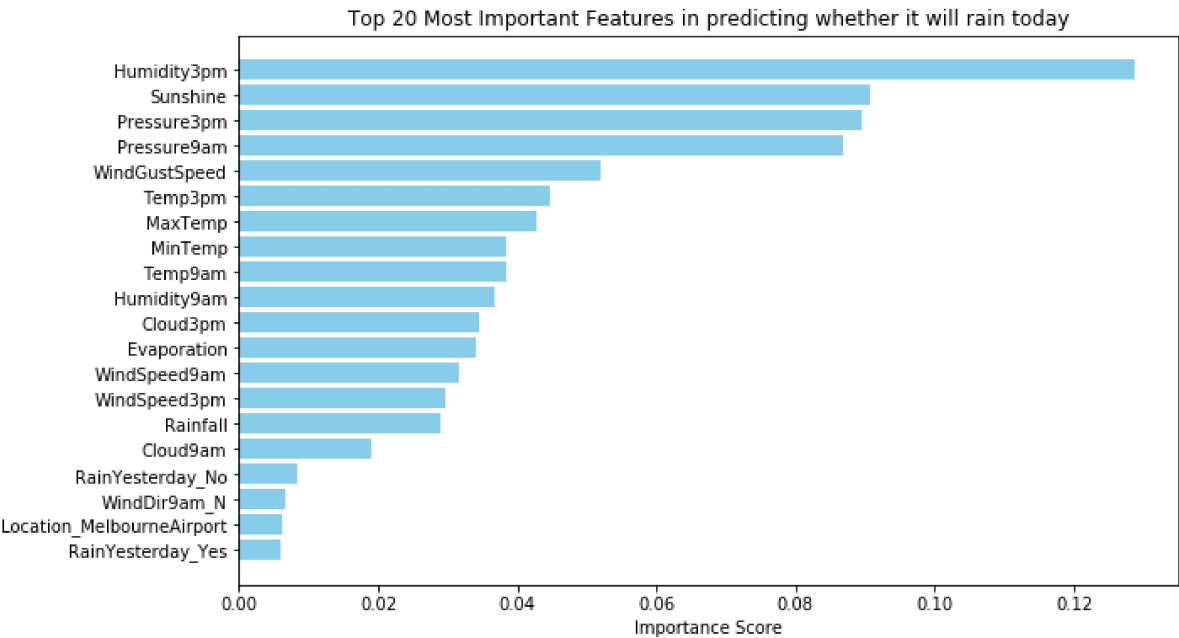
N = 20 # Change this number to display more or fewer features
top_features = importance_df.head(N)

# Plotting
plt.figure(figsize=(10, 6))
plt.barh(top_features['Feature'], top_features['Importance'], color='skyblue')
plt.gca().invert_yaxis() # Invert y-axis to show the most important feature o
n top
plt.title(f'Top {N} Most Important Features in predicting whether it will rain
today')
plt.xlabel('Importance Score')
plt.show()

# # Print test score
# test_score = model.score(X_test, y_test)
# print(f"\nTest set accuracy: {test_score:.2%}")
```

	Feature	Importance
9	Humidity3pm	0.128667
4	Sunshine	0.090627
11	Pressure3pm	0.089371
10	Pressure9am	0.086930
5	WindGustSpeed	0.052016
..	...	...
28	WindGustDir_SE	0.000862
51	WindDir3pm_E	0.000638
21	WindGustDir_ESE	0.000591
37	WindDir9am_ESE	0.000420
53	WindDir3pm_ESE	0.000280

[73 rows x 2 columns]



Point to note - 3

Identify the most important feature for predicting whether it will rain based on the feature importance bar graph. There will be a question on this in the assignment that follows.

Based on the updated feature importance bar graph for predicting whether it will rain today, the most important feature is "Humidity3pm," as it has the highest importance score.

Try another model

Some thoughts.

In practice you would want to try out different models and even revisit the data analysis to improve your model's performance. Maybe you can engineer better features, drop irrelevant or redundant ones, project your data onto a dimensional feature space, or impute missing values to be able to use more data. You can also try a larger set of parameters to define your search grid, or even engineer new features using cluster analysis. You can even include the clustering algorithm's hyperparameters in your search grid!

With Scikit-learn's powerful pipeline and GridSearchCV classes, this is easy to do in a few steps.

Exercise 15. Update the pipeline and the parameter grid

Let's update the pipeline and the parameter grid and train a Logistic Regression model and compare the performance of the two models. You'll need to replace the classifier with LogisticRegression. We have supplied the parameter grid for you.

Complete the following code:

```
# Replace RandomForestClassifier with LogisticRegression
pipeline.set_params(...=LogisticRegression(random_state=42))

# update the model's estimator to use the new pipeline
grid_search.estimator = ...

# Define a new grid with Logistic Regression parameters
param_grid = {
    # 'classifier__n_estimators': [50, 100],
    # 'classifier__max_depth': [None, 10, 20],
    # 'classifier__min_samples_split': [2, 5],
    'classifier__solver' : ['liblinear'],
    'classifier__penalty': ['l1', 'l2'],
    'classifier__class_weight' : [None, 'balanced']
}

grid_search.param_grid = ...

# Fit the updated pipeline with LogisticRegression
model.fit(..., ...)

# Make predictions
y_pred = model.predict(X_test)
```

```
In [362]: ## Write your response
# Replace RandomForestClassifier with LogisticRegression
pipeline.set_params(classifier=LogisticRegression(random_state=42))

# update the model's estimator to use the new pipeline
grid_search.estimator = pipeline

# Define a new grid with Logistic Regression parameters
param_grid = {
    # 'classifier__n_estimators': [50, 100],
    # 'classifier__max_depth': [None, 10, 20],
    # 'classifier__min_samples_split': [2, 5],
    'classifier__solver' : ['liblinear'],
    'classifier__penalty': ['l1', 'l2'],
    'classifier__class_weight' : [None, 'balanced']
}

grid_search.param_grid = param_grid

# Fit the updated pipeline with LogisticRegression
grid_search.fit(X_train, y_train)

# Make predictions
y_pred = grid_search.predict(X_test)
```

Fitting 5 folds for each of 4 candidates, totalling 20 fits

[CV] classifier__class_weight=None, classifier__penalty=l1, classifier__solver=liblinear

[CV] classifier__class_weight=None, classifier__penalty=l1, classifier__solver=liblinear, total= 0.2s

[Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent workers.

[Parallel(n_jobs=1)]: Done 1 out of 1 | elapsed: 0.1s remaining: 0.0s

```
[CV] classifier__class_weight=None, classifier__penalty=l1, classifier__solver=liblinear
[CV] classifier__class_weight=None, classifier__penalty=l1, classifier__solver=liblinear, total= 0.2s
[CV] classifier__class_weight=None, classifier__penalty=l1, classifier__solver=liblinear
[CV] classifier__class_weight=None, classifier__penalty=l1, classifier__solver=liblinear, total= 0.2s
[CV] classifier__class_weight=None, classifier__penalty=l1, classifier__solver=liblinear
[CV] classifier__class_weight=None, classifier__penalty=l1, classifier__solver=liblinear, total= 0.2s
[CV] classifier__class_weight=None, classifier__penalty=l1, classifier__solver=liblinear
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear, total= 0.1s
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear, total= 0.1s
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear, total= 0.0s
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear, total= 0.1s
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear
[CV] classifier__class_weight=None, classifier__penalty=l2, classifier__solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l1, classifier__solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__solver=liblinear
```



```

olver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__
solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__s
olver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__
solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__s
olver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__
solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__s
olver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__
solver=liblinear, total= 0.1s
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__s
olver=liblinear
[CV] classifier__class_weight=balanced, classifier__penalty=l2, classifier__
solver=liblinear, total= 0.1s
[Parallel(n_jobs=1)]: Done 20 out of 20 | elapsed: 2.2s finished

```

Compare the results to your previous model.

Display the clasification report and the confusion matrix for the new model and compare your results with the previous model.

```

In [363]: print(classification_report(y_test, y_pred))

from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
# Generate the confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='d')

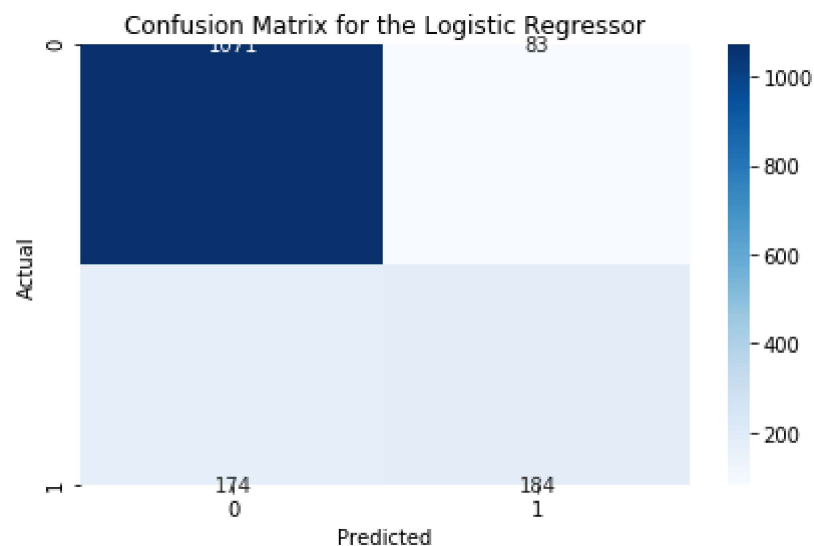
# Set the title and labels
plt.title('Confusion Matrix for the Logistic Regressor')
plt.xlabel('Predicted')
plt.ylabel('Actual')

# Show the plot
plt.tight_layout()
plt.show()

print(conf_matrix)

```

	precision	recall	f1-score	support
No	0.86	0.93	0.89	1154
Yes	0.69	0.51	0.59	358
accuracy			0.83	1512
macro avg	0.77	0.72	0.74	1512
weighted avg	0.82	0.83	0.82	1512



```

[[1071  83]
 [ 174 184]]

```

```
In [365]: # Extract True Positives (TP) and False Negatives (FN)
          TP = conf_matrix[1, 1]
          FN = conf_matrix[1, 0]

          # Calculate the True Positive Rate (TPR)
          TPR = TP / (TP + FN)

          print("True Positive Rate (TPR):", round(TPR*100), "%")
```

True Positive Rate (TPR): 51.0 %

What can you conclude about the model performances?

Points to note - 4

Compare the accuracy and true positive rate of rainfall predictions between the LogisticRegression model and the RandomForestClassifier model.

Note: Make sure to provide the answer in the form of a list using either bullets or numbers.

There will be a question on this in the assignment that follows.

1. Compare the accuracy percentages of both the classifiers.

Random Forest Classifier--->Accuracy: 0.84

Logistic Regression--->Accuracy: 0.84

2. Provide the details of the number of correct predictions.

Random Forest Classifier--->Correct predictions are 1271 out of 1512 total predictions, with an accuracy of approximately 84.4%.

Logistic Regression--->Correct predictions are 1277 out of 1512 total predictions, with an accuracy of approximately 84.4%.

3. Provide the true positive rate of LogisticRegression Classifier.

Random Forest Classifier--->True Positive Rate (TPR): 0.5223463687150838

Logistic Regression--->True Positive Rate (TPR): 0.5111731843575419

[Click here for Hints](#)

Congratulations! You've made it the end of your final project!

Well done! You now have some great tools to use for tackling complex real-world problems with machine learning.

Author

[Jeff Grossman \(https://www.linkedin.com/in/jpgrossman/\)](https://www.linkedin.com/in/jpgrossman/)

Other Contributor(s)

[Abhishek Gagneja \(https://www.linkedin.com/in/abhishek-gagneja-23051987/\)](https://www.linkedin.com/in/abhishek-gagneja-23051987/)

<!-- ## Changelog

| Date | Version | Changed by | Change Description |
|------------|---------|---------------|--------------------|
| 2024-11-26 | 0.1 | Jeff Grossman | Create lab |

-->

© IBM Corporation. All rights reserved.